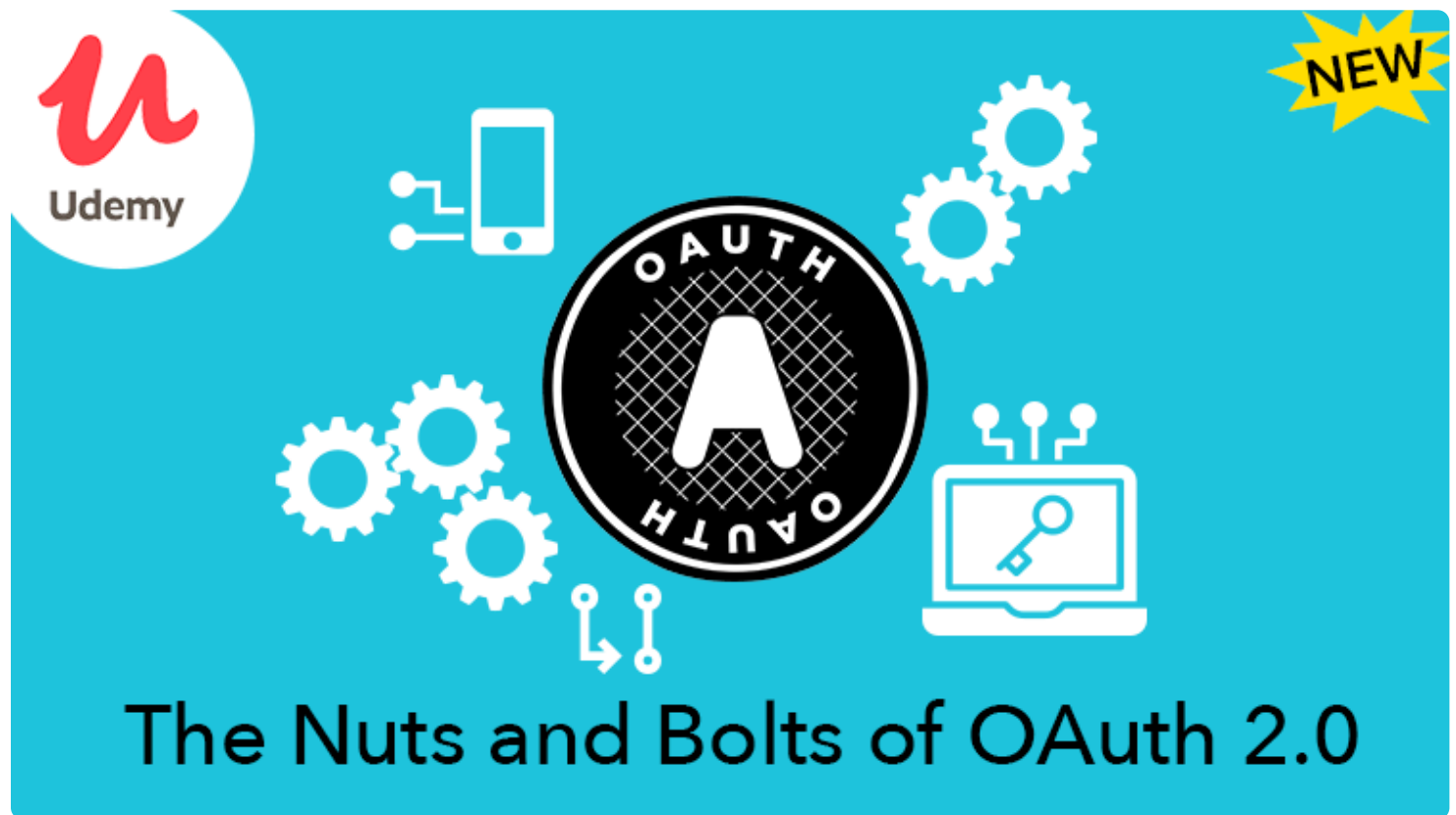


# Упрощенный OAuth 2

В этом посте в упрощенном формате описан протокол OAuth 2.0, чтобы помочь разработчикам и поставщикам услуг реализовать его.

Спецификация OAuth 2 может показаться немного запутанной, поэтому я написал этот пост, чтобы в упрощенном виде объяснить терминологию. В основной спецификации многие решения оставлены на усмотрение разработчика, часто с учетом компромиссов в вопросах безопасности. Вместо того чтобы описывать все возможные решения, которые необходимо принять для успешной реализации OAuth 2, в этом посте я расскажу о решениях, подходящих для большинства случаев.



Примечание. Этот пост был обновлен по сравнению с первоначальной версией 2012 года с учетом современных рекомендаций по использованию OAuth 2.0. С оригинальной версией можно ознакомиться [здесь](#).

## Содержание

- Роли: приложения, API и пользователи
- Создание приложения
- Авторизация: получение токена доступа
  - Приложения для веб-серверов

- Одностраничные приложения
- Мобильные приложения
- Другие типы грантов
- Отправка запросов с аутентификацией
- Отличия от OAuth 1.0
  - Аутентификация и подписи
  - Пользовательский опыт и альтернативные способы авторизации
  - Масштабируемая производительность
- Ресурсы

## Роли

### Стороннее приложение: "Клиент"

Клиент Ч это приложение, которое пытается получить доступ к учетной записи пользователя. Для этого ему необходимо получить разрешение пользователя.

### API: "Сервер ресурсов"

Сервер ресурсов Ч это сервер API, используемый для доступа к информации пользователя.

### Сервер авторизации

Это сервер, на котором представлен интерфейс, с помощью которого пользователь подтверждает или отклоняет запрос. В небольших системах это может быть тот же сервер, что и сервер API, но в более крупных развертываниях он часто выступает в качестве отдельного компонента.

### Пользователь: «Владелец ресурса»

Владелец ресурса Ч это человек, предоставляющий доступ к определенной части своей учетной записи.

## Создание приложения

Прежде чем приступить к процессу OAuth, необходимо зарегистрировать новое приложение в сервисе. При регистрации нового приложения обычно указывается основная информация, такая как название приложения, веб-сайт, логотип и т. д. Кроме того, необходимо указать URI перенаправления,

который будет использоваться для перенаправления пользователей на веб-сервер, в браузер или мобильное приложение.

## URI для перенаправления

Сервис будет перенаправлять пользователей только на зарегистрированный URI, что помогает предотвратить некоторые виды атак. Все URI для HTTP-перенаправления должны обслуживаться по протоколу HTTPS. Это помогает предотвратить перехват токенов в процессе авторизации. Нативные приложения могут зарегистрировать URI для перенаправления с помощью пользовательской схемы URL для приложения, которая может выглядеть как `demoapp://redirect`.

## Идентификатор и секретный ключ клиента

После регистрации приложения вы получите идентификатор клиента и, при необходимости, секретный ключ клиента. Идентификатор клиента считается общедоступной информацией и используется для создания URL-адресов для входа в систему или включается в исходный код Javascript на странице. Секретный ключ клиента **должен** храниться в секрете. Если развернутое приложение не может хранить секретный ключ в тайне, например одностраничные приложения на Javascript или нативные приложения, то секретный ключ не используется, и в идеале сервис вообще не должен выдавать секретный ключ таким приложениям.

## Авторизация

Первый этап OAuth 2 — получение авторизации от пользователя. В браузерных и мобильных приложениях это обычно делается путем отображения интерфейса, предоставляемого сервисом.

OAuth 2 предоставляет несколько «типов предоставления доступа» для различных сценариев использования. Определены следующие типы предоставления доступа:

- **Код авторизации** для приложений, работающих на веб-сервере, в браузере и мобильных приложениях
- **Пароль** для входа с использованием имени пользователя и пароля (только для собственных приложений)
- **Учетные данные клиента** для доступа к приложению без участия пользователя

- **Неявный метод** ранее рекомендовался для клиентов без секрета, но теперь его заменил метод авторизации с использованием кода авторизации и PKCE.

Каждый вариант использования подробно описан ниже.

## Приложения для веб-сервера

Веб-приложения — самый распространенный тип приложений, с которыми вы сталкиваетесь при работе с OAuth-серверами. Веб-приложения пишутся на серверном языке и запускаются на сервере, где исходный код приложения недоступен широкой публике. Это означает, что приложение может использовать свой клиентский ключ при взаимодействии с сервером авторизации, что позволяет избежать многих векторов атак.

### Авторизация

Создайте ссылку «Войти», которая будет перенаправлять пользователя на:

```
https://authorization-server.com/auth?response_type=code&
client_id=CLIENT_ID&redirect_uri=REDIRECT_URI&scope=photos&state=1234zyx
```

- **response\_type=code** — указывает на то, что ваш сервер ожидает получения кода авторизации
- **client\_id** — идентификатор клиента, который вы получили при первом создании приложения
- **redirect\_uri** — указывает URI, на который будет перенаправлен пользователь после авторизации
- **scope** — одно или несколько значений scope, указывающих, к каким частям учетной записи пользователя вы хотите получить доступ
- **state** — случайная строка, сгенерированная вашим приложением, которую вы проверите позже

Пользователь видит запрос на авторизацию



## An application would like to connect to your account

The app **Sample App** by Aaron Parecki would like the ability to access your basic information and photos.

Allow **Sample App** access?

Deny

Allow

Если пользователь нажимает «Разрешить», сервис перенаправляет его обратно на ваш сайт с кодом авторизации

```
https://example-app.com/cb?code=AUTH_CODE_HERE&state=1234zyx
```

- **код** Ч сервер возвращает код авторизации в строке запроса
- **состояние** Ч сервер возвращает то же значение состояния, которое вы указали

Сначала нужно сравнить это значение состояния, чтобы убедиться, что оно совпадает с исходным. Обычно значение состояния можно сохранить в файле cookie или в сеансе и сравнить его при повторном входе пользователя. Это поможет избежать попыток обмена произвольными кодами авторизации.

### Получение токена доступа

Ваш сервер обменивает код авторизации на токен доступа, отправляя POST-запрос на конечную точку токена сервера авторизации:

```
POST https://api.authorization-server.com/token
grant_type=authorization_code&
code=AUTH_CODE_HERE&
redirect_uri=REDIRECT_URI&
client_id=CLIENT_ID&
client_secret=CLIENT_SECRET
```

- **grant\_type=authorization\_code** Ч тип предоставления для этого потока Ч authorization\_code
- **code=AUTH\_CODE\_HERE** Ч код, который вы получили в строке запроса
- **redirect\_uri=REDIRECT\_URI** Ч должен совпадать с URI перенаправления, указанным в исходной ссылке
- **client\_id=CLIENT\_ID** Ч идентификатор клиента, который вы получили при первом создании приложения
- **client\_secret=CLIENT\_SECRET** Ч поскольку этот запрос выполняется из серверного кода, в нем указан секретный ключ

Сервер отвечает токеном доступа и указанием срока действия

```
{
  "access_token": "RsT50jbzRn430zqMLgV3Ia",
  "expires_in": 3600
}
```

или если произошла ошибка

```
{
  "error": "invalid_request"
}
```

Безопасность: обратите внимание, что сервис должен требовать от приложений предварительной регистрации URI перенаправления.

## Одностраничные приложения

Одностраничные приложения (или браузерные приложения) полностью запускаются в браузере после загрузки исходного кода с веб-страницы. Поскольку браузеру доступен весь исходный код, он не может обеспечить конфиденциальность клиентского секрета, поэтому в этом случае секрет не используется. Этот процесс основан на описанном выше процессе авторизации с использованием кода, но с добавлением динамически генерируемого секрета, который используется при каждом запросе. Это расширение называется **PKCE**.

Примечание. Ранее для браузерных приложений рекомендовалось использовать неявный поток, при котором токен доступа возвращается сразу после перенаправления и не требуется этап обмена токенами. С момента написания спецификации отраслевые стандарты изменились, и теперь рекомендуется использовать поток авторизации с кодом без секрета клиента. Это дает больше возможностей для создания безопасного потока, например с использованием

## Авторизация

Создайте случайную строку длиной от 43 до 128 символов, а затем сгенерируйте хэш SHA256 в кодировке base64, безопасный для использования в URL. Исходная случайная строка называется `code_verifier`, а хешированная версия — `code_challenge`.

Создайте случайную строку (верификатор кода), например  
5d2309e5bb73b864f989753887fe52f79ce5270395e25862da6940d5

Создайте хэш SHA256, а затем закодируйте строку в формате base64 (запрос кода): MChCW5vD-3h03HMGFZYskOSTir7II\_MMTb8a9rJNhnI

(Вы можете воспользоваться вспомогательной утилитой на [example-app.com/pkce](https://example-app.com/pkce), чтобы сгенерировать секретный ключ и хэш.)

Создайте ссылку «Войти», как в описанном выше процессе авторизации с помощью кода, но теперь включите в запрос запрос кода:

```
https://authorization-server.com/auth?response_type=code&  
client_id=CLIENT_ID&redirect_uri=REDIRECT_URI&scope=photos&state=1234zyx&code_challenge=
```

- **response\_type=code** - Указывает, что ваш сервер ожидает получения кода авторизации
- **client\_id** - идентификатор клиента, полученный при первом создании приложения.
- **redirect\_uri** - Указывает URI, к которому пользователь должен вернуться после завершения авторизации.
- **область действия** - Одно или несколько значений области действия, указывающих, к каким частям учетной записи пользователя вы хотите получить доступ.
- **state** - случайная строка, генерируемая вашим приложением, которую вы проверите позже.
- **code\_challenge** - Безопасный для URL-адреса хэш секрета SHA256 в кодировке base64, кодируемый по протоколу BASE64.
- **code\_challenge\_method=S256** - Укажите, какой метод хэширования вы использовали (S256)

Пользователь видит запрос на авторизацию



## An application would like to connect to your account

The app **Sample App** by Aaron Parecki would like the ability to access your basic information and photos.

Allow **Sample App** access?

Deny

Allow

Если пользователь нажимает «Разрешить», сервис перенаправляет его обратно на ваш сайт с кодом авторизации

```
https://example-app.com/cb?code=AUTH_CODE_HERE&state=1234zyx
```

- **код** Ч сервер возвращает код авторизации в строке запроса
- **состояние** Ч сервер возвращает то же значение состояния, которое вы указали

Сначала нужно сравнить это значение состояния, чтобы убедиться, что оно совпадает с исходным. Обычно значение состояния сохраняется в файле cookie, и при повторном входе пользователя в систему его можно сравнить с исходным. Это гарантирует, что конечную точку перенаправления не обманут и она не попытается обменять произвольные коды авторизации.

Полный пример использования PKCE в JavaScript можно найти в моем блоге [Is the OAuth Implicit Flow Dead?](#)

### Получение токена доступа

Теперь вам нужно обменять код авторизации на токен доступа, но вместо предварительно зарегистрированного клиентского секрета вы отправляете сгенерированный в начале процесса ключ PKCE.

```
POST https://api.authorization-server.com/token
grant_type=authorization_code&
```



```
code=AUTH_CODE_HERE&
redirect_uri=REDIRECT_URI&
client_id=CLIENT_ID&
code_verifier=CODE_VERIFIER
```

- **grant\_type=authorization\_code** Ч тип предоставления для этого потока Ч authorization\_code
- **code=AUTH\_CODE\_HERE** Ч код, который вы получили в строке запроса
- **redirect\_uri=REDIRECT\_URI** Ч должен совпадать с URI перенаправления, указанным в исходной ссылке
- **client\_id=CLIENT\_ID** Ч идентификатор клиента, который вы получили при первом создании приложения
- **code\_verifier=CODE\_VERIFIER** Ч случайный секретный ключ, который вы сгенерировали в начале

Сервер авторизации хеширует верификатор и сравнивает его с запросом, отправленным в запросе, и выдает токен доступа только в том случае, если они совпадают. Это гарантирует, что даже если кто-то перехватит код авторизации, он не сможет использовать его для получения токена доступа, поскольку у него не будет секретного ключа.

## Мобильные приложения

Как и браузерные приложения, мобильные приложения не могут обеспечить конфиденциальность клиентского секрета. Поэтому в мобильных приложениях также используется протокол PKCE, для которого не требуется клиентский секрет. Однако есть некоторые дополнительные аспекты, которые следует учитывать при разработке мобильных приложений, чтобы обеспечить безопасность протокола OAuth.

Примечание. Ранее для мобильных и нативных приложений рекомендовалось использовать неявное предоставление прав. С момента написания спецификации отраслевые стандарты изменились, и теперь для нативных приложений рекомендуется использовать поток авторизации без секрета. Для нативных приложений также есть [дополнительные рекомендации](#), с которыми стоит ознакомиться.

## Авторизация

Создайте кнопку «Войти», которая будет перенаправлять пользователя либо в нативное приложение сервиса на телефоне, либо на мобильную веб-страницу сервиса. Приложения могут зарегистрировать пользовательскую схему URI, например «example-app://», чтобы нативное приложение

запускалось при переходе по URL с таким протоколом, или зарегистрировать шаблоны URL, которые будут запускать нативное приложение при переходе по URL, соответствующему шаблону.

#### Использование нативного приложения сервиса

Если у пользователя установлено нативное приложение Facebook, перенаправьте его по следующему URL-адресу:

```
fbauth2://authorize?response_type=code&client_id=CLIENT_ID
&redirect_uri=REDIRECT_URI&scope=email&state=1234zyx
```

- **response\_type=code** - указывает, что ваш сервер ожидает получения кода авторизации
- **client\_id=CLIENT\_ID** - идентификатор клиента, который вы получили при первом создании приложения.
- **redirect\_uri=REDIRECT\_URI** - Указывает URI, к которому следует вернуть пользователя после завершения авторизации, например fb000000000://authorize
- **область действия= электронная почта** - Одно или несколько значений области действия, указывающих, к каким частям учетной записи пользователя вы хотите получить доступ.
- **state=1234zyx** - Случайная строка, сгенерированная вашим приложением, которую вы проверите позже

Для серверов, поддерживающих [расширение PKCE](#) (а если вы создаете сервер, то он должен поддерживать расширение PKCE), необходимо указать следующие параметры. Сначала создайте «верификатор кода» Ч случайную строку, которую приложение хранит локально.

- **code\_challenge=XXXXXXXX** Ч это закодированная в формате base64 версия хэша sha256 строки верификатора кода
- **code\_challenge\_method=S256** Ч указывает метод хеширования, используемый для вычисления контрольной суммы, в данном случае Ч sha256.

Обратите внимание, что ваш URI перенаправления, скорее всего, будет выглядеть как fb000000000://authorize, где protocol Ч это пользовательская схема URL, которую ваше приложение зарегистрировало в операционной системе.

#### Использование веб-браузера

Если у сервиса нет собственного приложения, вы можете запустить мобильный браузер и перейти по стандартному URL-адресу для веб-авторизации. Обратите внимание, что ни в коем случае нельзя использовать встроенное веб-представление в собственном приложении, так как это не гарантирует, что пользователь вводит пароль на сайте сервиса, а не на фишинговом сайте.

Вам нужно либо запустить встроенный мобильный браузер, либо использовать новый iOS-компонент `SafariViewController` для запуска встроенного браузера в вашем приложении. Этот API был добавлен в iOS 9 и представляет собой механизм запуска браузера внутри приложения, который отображает адресную строку, чтобы пользователь мог убедиться, что он находится на нужном сайте, а также позволяет обмениваться файлами cookie с настоящим браузером Safari. Кроме того, он не позволяет приложению проверять и изменять содержимое браузера, поэтому его можно считать безопасным.

```
https://facebook.com/dialog/oauth?response_type=code&client_id=CLIENT_ID  
&redirect_uri=REDIRECT_URI&scope=email&state=1234zyx
```

Опять же, если сервис поддерживает протокол PKCE, то эти параметры следует указать вместе с описанными выше.

- **response\_type=code** - указывает, что ваш сервер ожидает получения кода авторизации
- **client\_id=CLIENT\_ID** - идентификатор клиента, который вы получили при первом создании приложения.
- **redirect\_uri=REDIRECT\_URI** - Указывает URI, к которому следует вернуть пользователя после завершения авторизации, например `fb00000000://authorize`
- **область действия= электронная почта** - Одно или несколько значений области действия, указывающих, к каким частям учетной записи пользователя вы хотите получить доступ.
- **state=1234zyx** - Случайная строка, сгенерированная вашим приложением, которую вы проверите позже

Пользователь увидит запрос на авторизацию



## Получение токена доступа

После нажатия кнопки «Подтвердить» пользователь будет перенаправлен обратно в ваше приложение по URL-адресу, например

```
fb00000000://authorize?code=AUTHORIZATION_CODE&state=1234zyx
```

Ваше мобильное приложение должно сначала проверить, соответствует ли состояние устройства состоянию, указанному в первоначальном запросе, а затем обменять код авторизации на токен доступа.

Обмен токенами будет происходить так же, как обмен кодом в приложении на веб-сервере, за исключением того, что секретный ключ не будет отправлен. Если сервер поддерживает протокол PKCE, вам нужно будет указать дополнительный параметр, как описано ниже.

```
POST https://api.authorization-server.com/token
grant_type=authorization_code&
code=AUTH_CODE_HERE&
redirect_uri=REDIRECT_URI&
```

```
client_id=CLIENT_ID&  
code_verifier=VERIFIER_STRING
```

- **grant\_type=authorization\_code** Ч тип предоставления для этого потока Ч authorization\_code
- **code=AUTH\_CODE\_HERE** Ч код, который вы получили в строке запроса
- **redirect\_uri=REDIRECT\_URI** Ч должен совпадать с URI перенаправления, указанным в исходной ссылке
- **client\_id=CLIENT\_ID** Ч идентификатор клиента, который вы получили при первом создании приложения
- **code\_verifier=VERIFIER\_STRING** Ч строка в открытом виде, которую вы ранее хешировали для создания code\_challenge

The authorization server will verify this request and return an access token.

Если сервер поддерживает протокол PKCE, то сервер авторизации распознает, что этот код был сгенерирован с помощью запроса на код, хеширует предоставленный открытый текст и подтверждает, что хешированная версия соответствует хешированной строке, отправленной в первоначальном запросе на авторизацию. Это обеспечивает безопасность использования потока авторизации с помощью кода для клиентов, которые не поддерживают секретные данные.

## Другие типы предоставления доступа

### Пароль

В OAuth 2 также предусмотрен тип предоставления доступа «пароль», который позволяет напрямую обменять имя пользователя и пароль на токен доступа. Поскольку для этого приложению необходимо получить пароль пользователя, этот тип предоставления доступа может использоваться только приложениями, созданными самим сервисом. Например, встроенное приложение Twitter может использовать этот тип предоставления доступа для входа в мобильные и десктопные приложения.

Чтобы использовать тип предоставления доступа «пароль», просто отправьте POST-запрос следующего вида:

```
POST https://api.authorization-server.com/token  
grant_type=password&  
username=USERNAME&  
password=PASSWORD&  
client_id=CLIENT_ID
```

- **grant\_type=password** Ч тип предоставления доступа для этого потока Ч пароль
- **username=USERNAME** Ч имя пользователя, полученное от приложения
- **password=PASSWORD** Ч пароль пользователя, полученный от приложения
- **client\_id=CLIENT\_ID** Ч идентификатор клиента, полученный при создании приложения

Сервер отвечает токеном доступа в том же формате, что и при других типах предоставления доступа.

Обратите внимание, что здесь не указан секретный ключ клиента, поскольку предполагается, что в большинстве случаев для предоставления доступа по паролю будут использоваться мобильные или настольные приложения, в которых секретный ключ не может быть защищен.

## Доступ к приложению

В некоторых случаях приложениям может потребоваться токен доступа, чтобы действовать от своего имени, а не от имени пользователя. Например, сервис может предоставить приложению возможность обновлять собственную информацию, такую как URL-адрес веб-сайта или значок, или получать статистику о пользователях приложения. В этом случае приложениям нужен способ получить токен доступа для своей учетной записи без привязки к конкретному пользователю. Для этого в OAuth предусмотрен тип предоставления `client_credentials` доступа.

Чтобы использовать тип предоставления доступа на основе учетных данных клиента, отправьте POST-запрос следующего вида:

```
POST https://api.authorization-server.com/token
grant_type=client_credentials&
client_id=CLIENT_ID&
client_secret=CLIENT_SECRET
```

В ответе будет указан токен доступа в том же формате, что и при других типах предоставления доступа.

## Отправка запросов с аутентификацией

Конечным результатом всех типов предоставления доступа является получение токена доступа.

Теперь, когда у вас есть токен доступа, вы можете отправлять запросы к API. Вы можете быстро отправить запрос к API с помощью cURL следующим образом:

```
curl -H "Authorization: Bearer RsT50jbzRn430zqMLgV3Ia" \
https://api.authorization-server.com/1/me
```

Вот и всё! Убедитесь, что вы всегда отправляете запросы по протоколу HTTPS и никогда не игнорируете недействительные сертификаты. HTTPS — единственная технология, которая защищает запросы от перехвата и изменения.

## Отличия от OAuth 1.0

Протокол OAuth 1.0 во многом основывался на существующих проприетарных протоколах, таких как FlickrAuth от Flickr и AuthSub от Google. В результате было получено оптимальное решение, основанное на реальном опыте внедрения. Однако за несколько лет работы с протоколом сообщество накопило достаточно знаний, чтобы переосмыслить и усовершенствовать его в трех основных областях, где OAuth 1.0 оказался недостаточно гибким или запутанным.

Подробнее об этом можно прочитать в моей книге [OAuth 2.0 для начинающих](#).

## Ресурсы

- OAuth 2.0 для начинающих — книга [oauth2simplified.com](#)
- Узнайте больше о [создании серверов OAuth 2.0](#)
- [Расширение PKCE](#)
- [Рекомендации для нативных приложений](#)
- Более подробная информация доступна на [OAuth.net](#)
- Некоторые материалы адаптированы с [hueniverse.com](#).

Предыдущие версии этого поста:

 Чашка для чаевых 

- [Июль 2012 года](#)

### Об авторе



Аарон Пареки — автор книги [OAuth 2.0 Simplified](#), администратор сайта [oauth.net](#) и редактор нескольких спецификаций W3C.

Хотите, чтобы Аарон Пареки выступил на вашей встрече с докладом об OAuth?  
Запросите презентацию у Аарона.

Постоянная ссылка



Привет, я Аарон, директор по стандартам идентификации в компании Okta и соучредитель IndiewebCamp. Я веду [oauth.net](#), пишу и консультирую по вопросам OAuth, а также участвую в рабочей группе по OAuth в IETF. Кроме того, я помогаю людям узнать больше о производстве видео и прямых трансляциях. (подробная биография)

Я отслеживаю свое местоположение с 2008 года и написал 100 песен за 100 дней. Я выступал на конференциях по всему миру с докладами о власти над своими данными, OAuth, квантифицированном «я» и объяснял, почему буква R Ч гласная. Читать дальше.

- Директор по стандартам идентификации в Okta
- IndiewebCamp Основатель
- Рабочая группа по OAuth Редактор
- OpenID Член правления
- Обучающие видео и обзоры на YouTube
- Мы строим триплекс!
- Life Stack
- Домашняя автоматизация

Поиск...

Все Статьи Закладки Заметки Фотографии Ответы Отзывы Путешествия Видео Контакты

© 1999ц2026 Аарон Пареcki. Создано с помощью p3k. Этот сайт поддерживает Webmention.  
Except where otherwise noted, text content on this site is licensed under a Creative Commons Attribution 3.0 License.